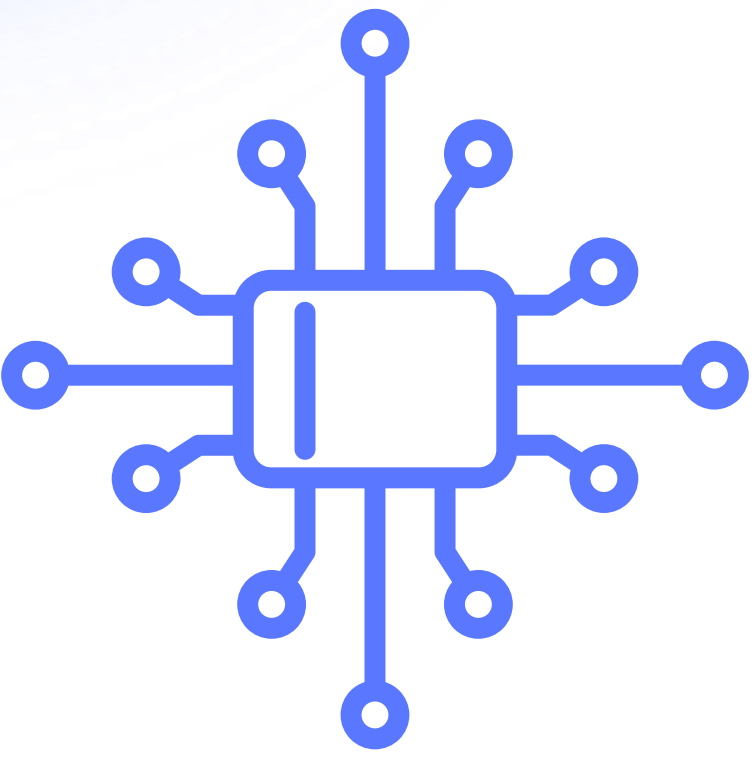
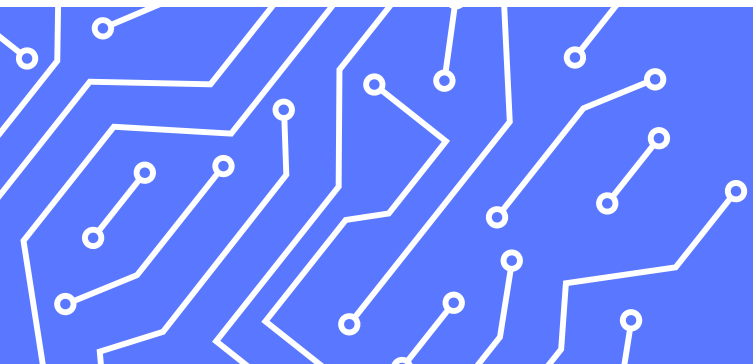
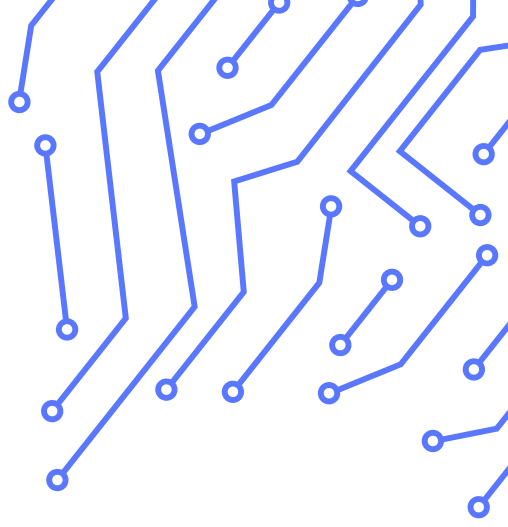




RECURRENT NEURAL NETWORKS (RNN'S) AND LONG SHORT-TERM MEMORY (LSTM) FOR SEQUENCE PROCESSING

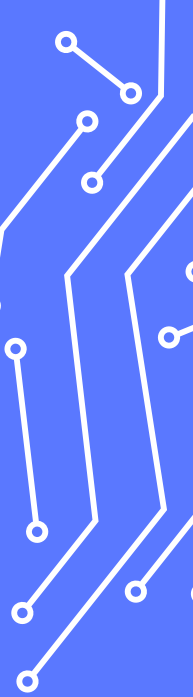


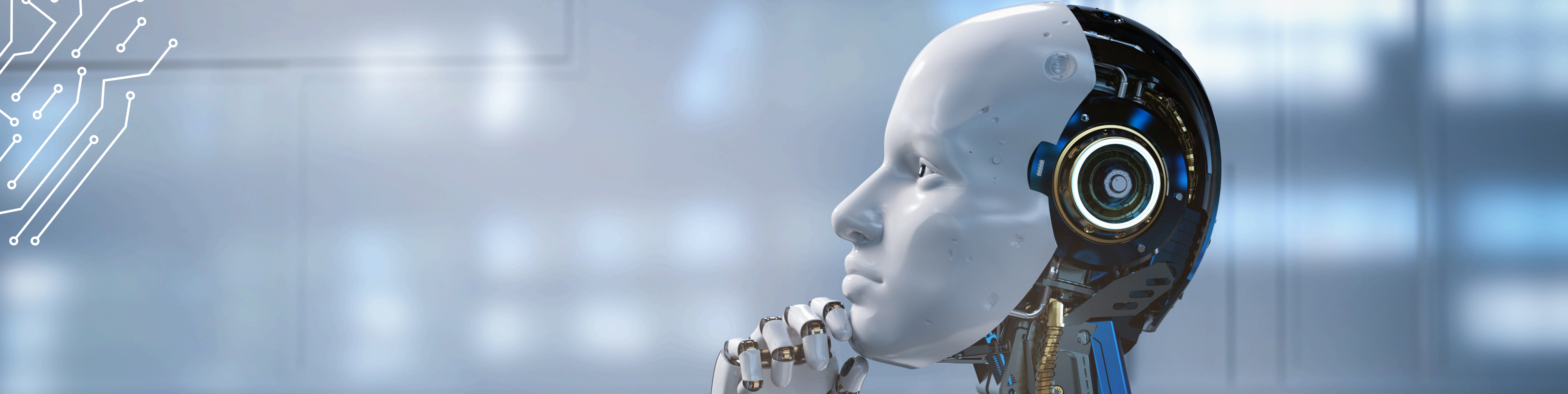
BACKGROUND

Recurrent Neural Networks emerged as a solution for processing sequential data.

Unlike traditional feed-forward neural networks, RNNs maintain information from previous time steps.

This makes them useful when the order and context of the data are important.

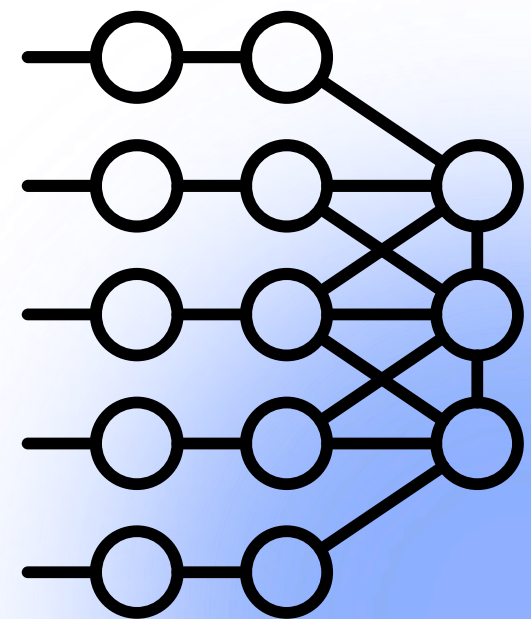




WHAT PROBLEMS CAN RNNs AND LSTMS SOLVE?

RNNs and LSTMs can be applied to:

- Natural language processing
- Speech and audio processing
- Time-series forecasting
- Sensor data analysis
- Music and signal processing
- Sequence classification



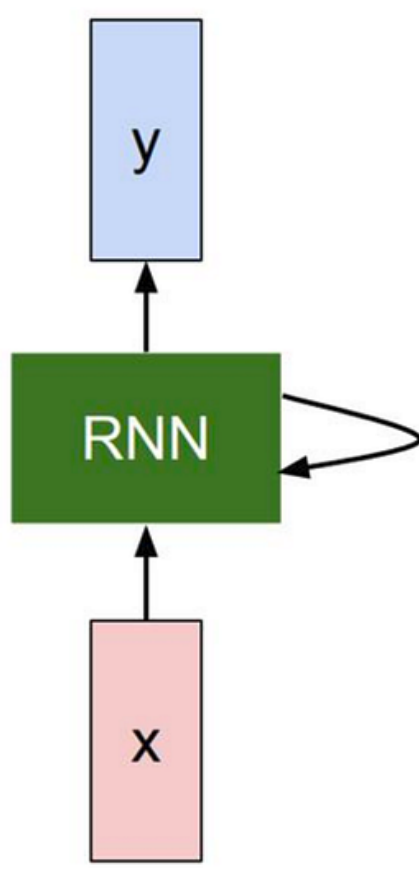
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



RNN ARCHITECTURE

Recurrent Neural Networks emerged as a solution for processing sequential data.

Unlike traditional feed-forward neural networks, RNNs maintain information from previous time steps.

This makes them useful when the order and context of the data are important.

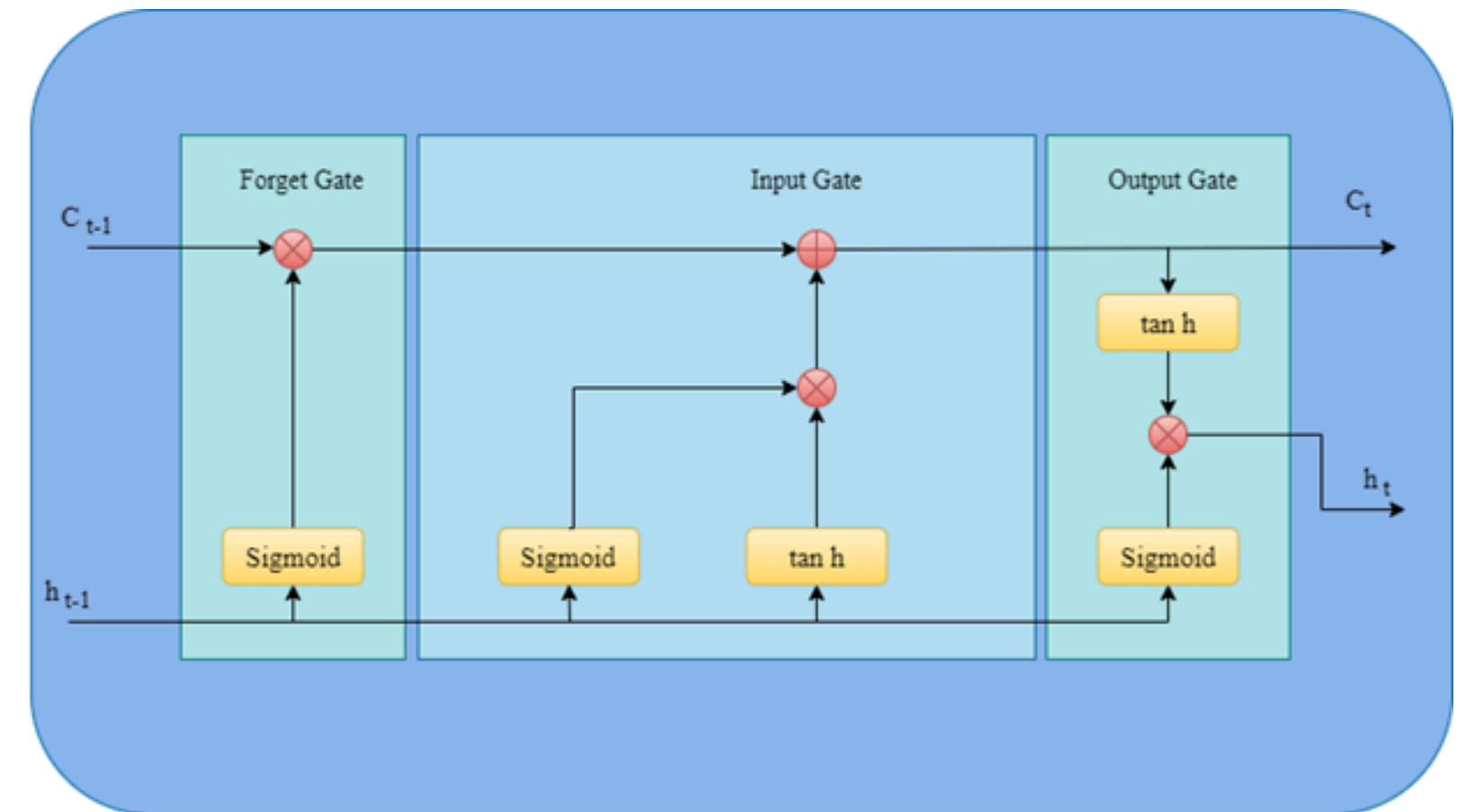


LONG SHORT-TERM MEMORY (LSTM)

LSTM is a type of RNN designed to better handle long-term dependencies.

It uses gates to control the flow of information:

- Forget Gate
 - Decides what information to discard.
- Input Gate
 - Decides what new information to store.
- Cell State
 - Carries information through the sequence.
- Output Gate
 - Determines what information is used as output.
- Candidate Memory
 - Proposes new information that can be added to the cell state.



3 gates:

- Forget gate, Input gate, Output gate

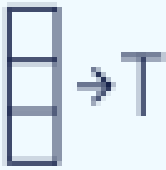
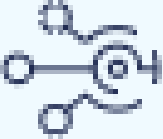
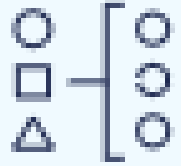
3 states/components

- Cell state, Hidden state, Candidate memory



ACTIVATION FUNCTIONS & NON-LINEARITY

Non-linearity allows the network to learn complex curved patterns

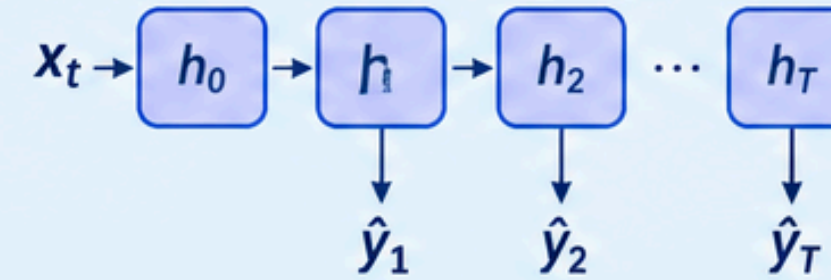
Network Type	 Standard RNN	 LSTM	 Classification Output
Activation Functions Used	Tanh or ReLU	Tanh and Sigmoid	Softmax

ReLU is commonly used in other neural network architectures.

Learning and Optimization Mechanism

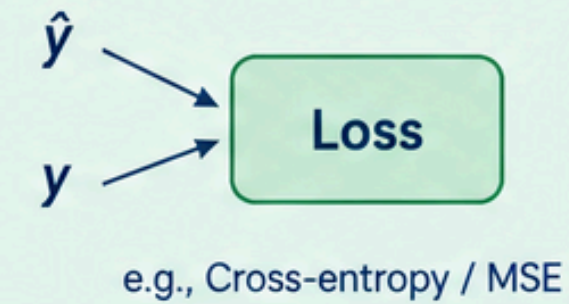
1 Forward Propagation

- The sequence is processed step by step.
- At each time step, the network updates its internal state and generates a prediction.
- In an RNN/LSTM, the hidden state carries information from previous steps.



2 Loss Function

- The predicted output ($\hat{y}$) is compared with the real value (y).
- The loss tells us how wrong the prediction is.
- Examples: cross-entropy (classification) or MSE (regression).



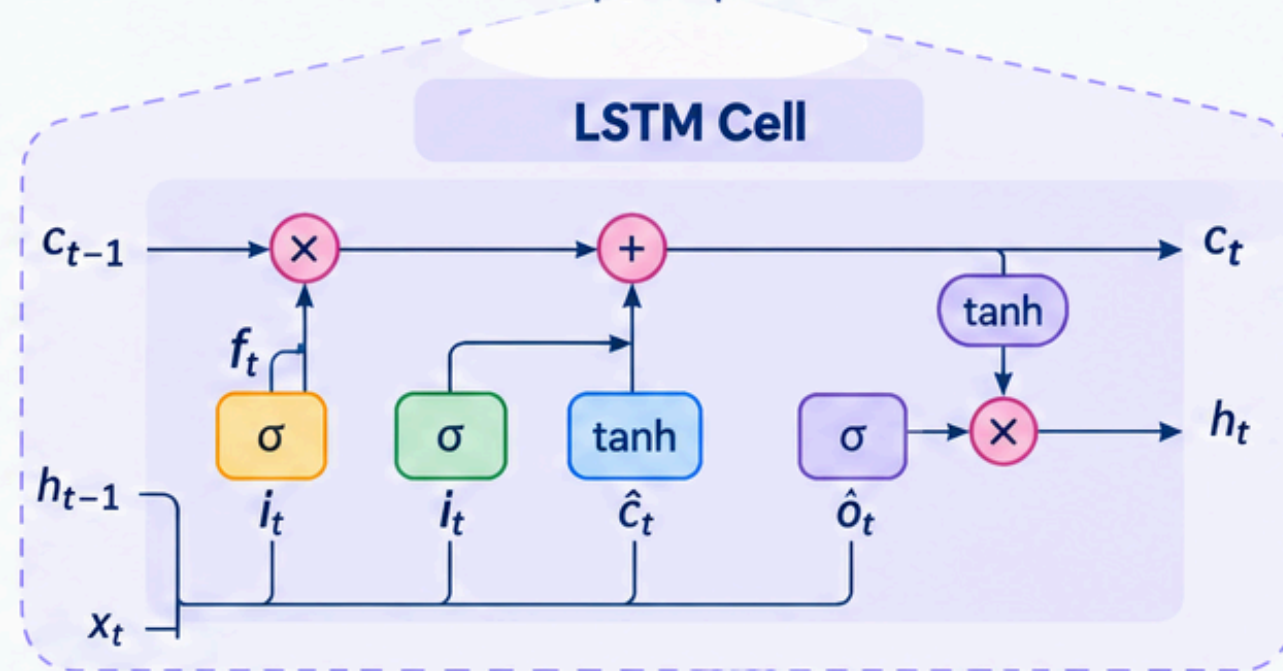
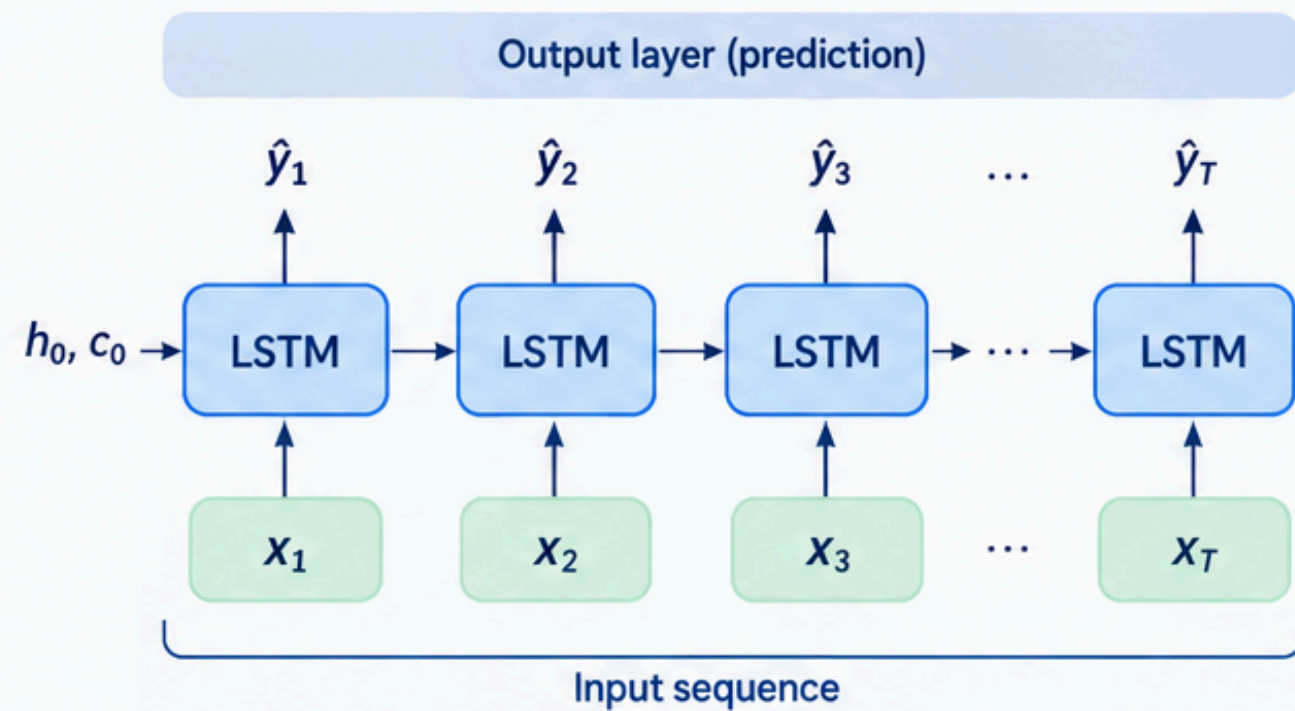
3 Backpropagation and Optimization

- The gradients of the loss with respect to the parameters are calculated.
- An optimizer (e.g., gradient descent or Adam) updates the weights to minimize the error.
- The process is repeated for several epochs.

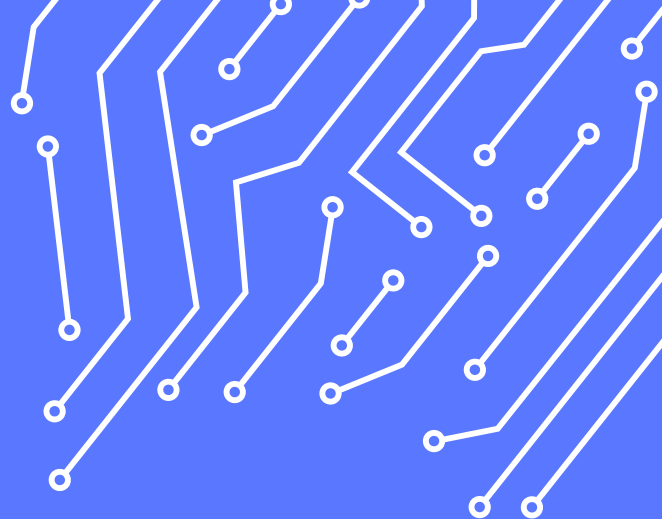


Repeat for multiple epochs

LSTM Architecture

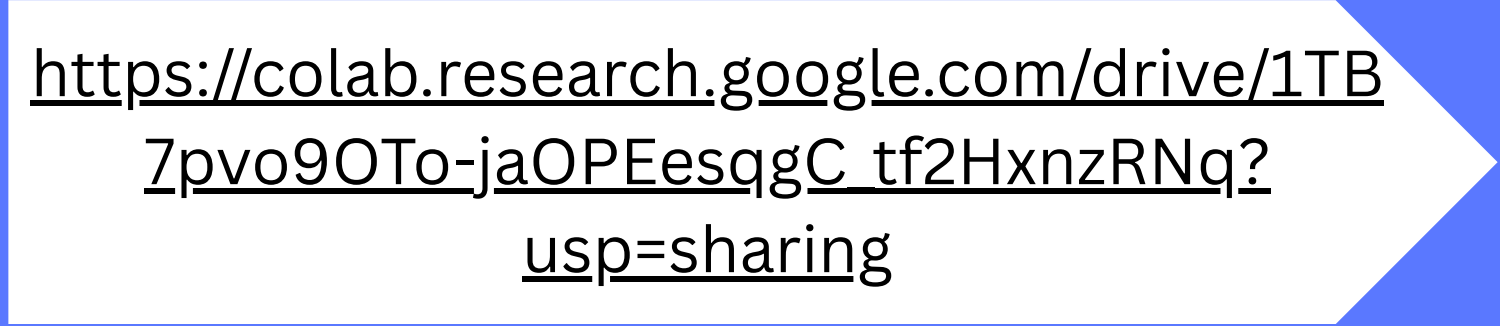


- | | |
|---------------------------------------|------------------------------------|
| σ = sigmoid function (0 - 1) | $\times$ = multiplication (gating) |
| $\tanh$ = hyperbolic tangent (-1 - 1) | $+$ = sum (state update) |

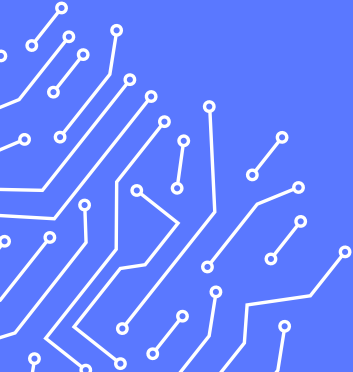


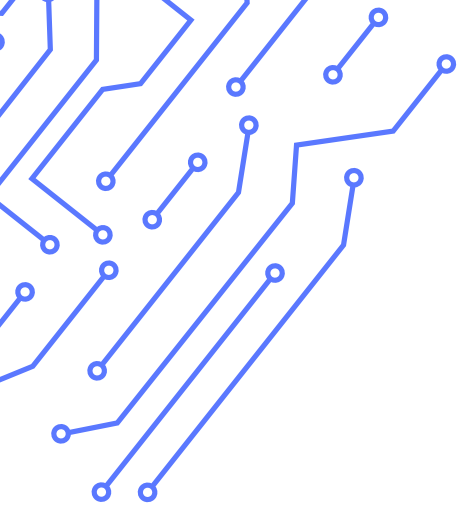
IMPLEMENTACIÓN

PYTHON / PYTORCH
RNN AND LSTM SEQUENCE CLASSIFICATION



https://colab.research.google.com/drive/1TB7pvo90To-jaOPEesqgC_tf2HxnzRNq?usp=sharing



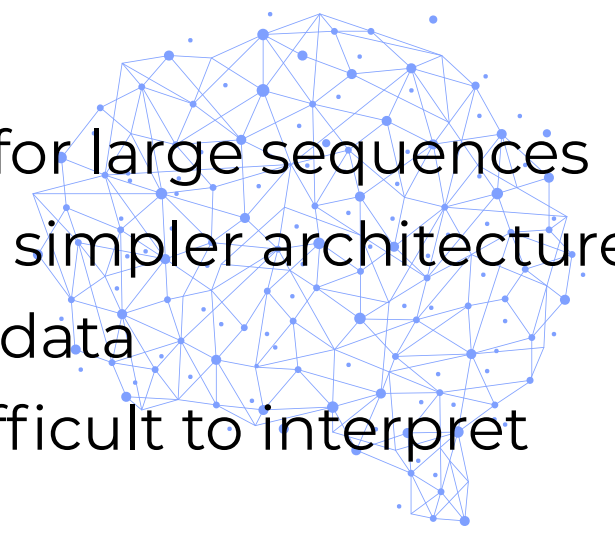


LIMITATIONS AND CONCLUSION

Strengths

- Handles sequential data
- Maintains information across time steps
- Useful for temporal dependencies
- Applicable to text, audio and time series

Limitations

- Higher computational cost for large sequences
 - Training can be slower than simpler architectures
 - Requires sufficient training data
 - Internal decisions can be difficult to interpret
- 

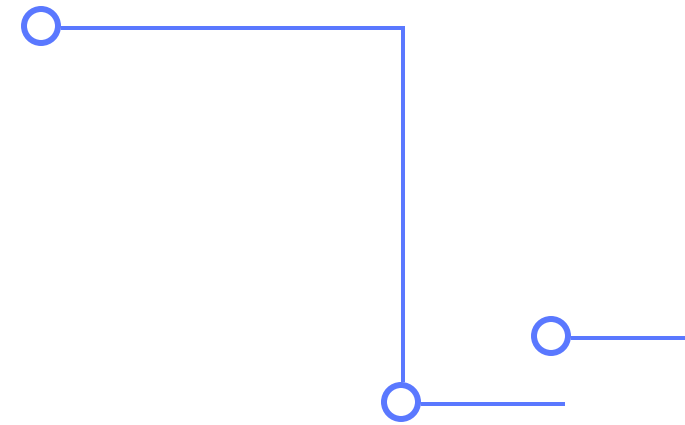
RNNs are designed to process sequential data by maintaining information from previous time steps.

LSTMs improve this approach by using a cell state and gates to control what information is forgotten, stored, and passed to the output.

Their ability to capture temporal dependencies makes them useful for applications such as language, speech, time series, and sequence classification. However, their greater computational cost and complexity must be considered when choosing an architecture.

REFERENCES

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [3] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [4] C. Olah, “Understanding LSTM Networks,” *colah's blog*, 2015.
- [5] PyTorch, “LSTM — PyTorch Documentation,” *PyTorch Documentation*.



THANK YOU